If a subclass changes the visibility of a method in a parent, Ruby effectively inserts a hidden proxy method in the subclass that invokes the original method using super. It then sets the visibility of that proxy to whatever you requested. This means that the following code:

```
class KeepItPrivate < Base
  public :a_method
end
```

is effectively the same as this:

```
class KeepItPrivate < Base
  def a_method(*)
    super
  end
  public :a_method
end
```

The call to super can access the parent's method regardless of its visibility, so the rewrite allows the subclass to override its parent's visibility rules.

Modules and Mixins

You know that when you include a module into a Ruby class, the instance methods in that module become available as instance methods of the class, like this:

```
module Logger
  def log(msg)
    STDERR.puts Time.now.strftime("%H:%M:%S: ") + "#{self} (#{msg})"
  end
end

class Song
  include Logger
end

s = Song.new
s.log("created")
```

produces:

```
12:30:30: #<Song:0x0000000100e38680> (created)
```

Ruby implements include very simply: The module you include is added as a ancestor of the class being defined. It's as if the module is the parent of the class that it is mixed in to. And that would be the end of the description except for one small wrinkle. Because the module is injected into the chain of superclasses, it must itself hold a link to the original parent class. If it didn't, there'd be no way of traversing the superclass chain to look up methods. However, you can mix the same module into many different classes, and those classes could potentially have totally different superclass chains. If there were just one module object that we mixed in to all these classes, there'd be no way of keeping track of the different superclasses for each.

To get around this, Ruby uses a clever trick. When you include a module in class Example, Ruby constructs a new class object, makes it the superclass of Example, and then sets the superclass of the new class to be the original superclass of Example. It then references the module's methods from this new class object in such a way that when you look a method up in this class, it actually looks it up in the module: